

IF2224: Teori Bahasa Formal dan Automata

**Pembuatan Lexical Analyser
Untuk Bahasa Pemrograman Arion**



Dipersiapkan oleh:

13524008	Agatha Tatianingseto
13524030	Irvin Tandiarrang Sumual
13524074	Bernhard Aprillio Pramana
13524096	Moreno Syawali Ganda Sugita
13524110	Jennifer Khang

**PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
JL. GANESA 10, BANDUNG 40132
2025**

Bab 1

Deskripsi Tugas

Bahasa pemrograman adalah cara agar kita dapat menuliskan program komputer tanpa perlu menghadapi sulitnya menulis dalam bahasa mesin. Bahasa mesin (biner 1 dan 0) adalah satu-satunya bahasa yang dapat dipahami dan dijalankan oleh CPU, namun sangat sulit dibaca oleh manusia. Oleh karena itu, bahasa pemrograman sering kali dibedakan dengan istilah low-level atau high-level. High-level artinya bahasa lebih mendekati bahasa manusia dan sebaliknya low-level lebih mendekati bahasa mesin. Semakin mendekati bahasa manusia, maka akan semakin banyak proses yang diperlukan untuk mengubahnya menjadi kode mesin.

Terdapat dua macam cara mengubah/memproses bahasa pemrograman menjadi machine code, salah satunya adalah **compiler**. Interpreter membaca, menerjemahkan, dan langsung mengeksekusi kode mesin baris per baris sehingga jika terdapat error pada baris ke 10, program akan tetap berjalan normal untuk 9 baris awal, dibandingkan dengan compiler yang akan gagal dijalankan jika terdeteksi terdapat error di dalam source code. Compiler secara umum terdiri dari 4 tahapan, mulai dari Lexical Analysis, Syntax Analysis, Semantic Analysis hingga Intermediate Code Generation.

Dalam konteks tugas ini, **Arion Compiler - Lexical Analysis** merupakan bagian dari pengembangan sebuah compiler untuk bahasa pemrograman Arion, khususnya pada tahap awal yaitu lexical analysis. Program yang dibuat harus mampu membaca file input berupa source code bahasa Arion dalam bentuk file .txt dan menghasilkan output berupa daftar token yang terstruktur. Output ini akan menjadi dasar bagi tahap berikutnya dalam pengembangan interpreter, sehingga keakuratan dan kelengkapan hasil tokenisasi menjadi aspek penting dalam implementasi lexer ini.

Bab 2

Teori Singkat

2.1. Pengertian dan Fungsi Lexical Analyzer (Lexer)

Lexical Analyzer adalah tahap pertama dalam proses kompilasi atau interpretasi bahasa pemrograman. Pada tahap ini, *compiler* memecah sintaks-sintaks menjadi token yang berurutan, dengan menghapus spasi dan komentar pada sebuah *source code*. Tahap ini juga berguna untuk mendeteksi token yang tidak valid dan menghasilkan *error* sebelum program tersebut diproses lebih lanjut ^[1].

2.2. Peran Lexer dan Proses Kompilasi

Tujuan dari *Lexical Analyzer* adalah untuk mempermudah pemrosesan *source code* pada tahap berikutnya, yaitu tahap *parsing*. Tahap *parsing* dapat bekerja dengan memproses kumpulan token yang sudah terurut daripada memproses karakter mentah langsung dari *source code*, sehingga lebih mudah memahami alur logika dari program yang di-*compile* ^[2].

2.3. Konsep Token dan Deterministic Finite Automata (DFA)

Token merupakan unit terkecil yang memiliki arti dalam sebuah program, seperti operator, angka, tanda baca, kata kunci, dan variabel. Proses pembentukan token dilakukan oleh Lexer yang membaca *source code* dari awal hingga akhir dan mengenali setiap urutan karakter untuk mengenali jenis token yang sesuai. Agar proses proses pembacaan dan pengenalan pola pada *source code* dapat dijalankan secara otomatis oleh komputer, dapat diimplementasikan dalam bentuk *Finite Automata*, khususnya *Deterministic Finite Automata* (DFA) ^[3].

Bab 3

Perancangan & Implementasi

3.1. Diagram Transisi DFA

Diagram transisi DFA dirancang menggunakan *workspace* berbasis web draw.io untuk merepresentasikan alur logika perpindahan *state* untuk mengenali seluruh token sekaligus menghasilkan *error* jika terdapat kesalahan token pada tahap *Lexical Analysis*. Diagram ini mencakup seluruh kategori token yang diperlukan. Struktur diagram memiliki bentuk *Deterministic Finite Automata* (DFA) untuk memastikan setiap urutan karakter yang diproses menuju *state* yang tepat. Hasil diagram transisi yang telah dirancang dapat diakses pada tautan berikut serta pada bagian Lampiran.

 Diagram Transisi DFA.png

3.2. Perancangan Program

Program dikembangkan menggunakan bahasa pemrograman C++ GNU dengan sistem kompilasi berbasis MAKEFILE untuk efisiensi pengembangan ke depannya. Alur kerja program meliputi:

- Menerima masukan berupa *source code* dalam format *.txt*.
- Melakukan pemrosesan karakter per karakter sesuai dengan logika DFA yang telah dirancang.
- Menghasilkan keluaran berupa daftar token (serta *value* jika diperlukan) dan *UNKNOWN* jika ditemukan kesalahan identifikasi token yang disimpan dalam *file .txt*.
- Pemilihan ketentuan *comment* adalah *escape symbol* harus sama dengan *start symbol* nya, sehingga { hanya akan bisa ditutup oleh }, begitu juga dengan (* dan *).

3.3. Implementasi Program

Implementasi kode program merupakan hasil translasi dari diagram transisi DFA menjadi bahasa pemrograman C++. Implementasi ini bekerja dengan membaca karakter

satu per satu yang menggerakkan transisi antar *state* berdasarkan karakter yang diterima. *Error handling* juga diimplementasikan untuk memberikan pesan kesalahan yang sesuai jika ditemukan simbol yang tidak dikenali oleh DFA agar program tidak *crash* saat dilanjutkan pada tahap berikutnya. Hasil implementasi program dapat diakses pada tautan repositori Github yang terlampir pada bagian Lampiran.

Bab 4

Pengujian

Pengujian 1

1. Nama Pengujian : Case Insensitive
2. Input file : case_insensitive.txt
3. Output file : case-insensitive_result.txt
4. Deskripsi :

Test Case ini dibuat untuk memastikan bahwa lexer bersifat case-insensitive dalam mengenali keyword, sehingga berbagai variasi huruf besar/kecil tetap dikenali sebagai keyword yang sama.

5. Hasil Pengujian :

Tabel 4.1 Pengujian 1 : Case-Insensitive

Input	Output
<pre>PrOgRaM MiXeDcAsEtEsT; VaR Number1, xValue, ReAl123: integer; BeGiN Number1 := 12; IF Number1 == 12 ThEn xValue := Number1 + 3; eLsE ReAl123 := 0; EnD.</pre>	<pre>programsy ident (MiXeDcAsEtEsT) semicolon varsy ident (Number1) comma ident (xValue) comma ident (ReAl123) colon ident (integer) semicolon beginsy ident (Number1) becomes intcon (12) semicolon ifsy</pre>

	<pre>ident (Number1) eq1 intcon (12) thensy ident (xValue) becomes ident (Number1) plus intcon (3) semicolon elsesy ident (ReAl123) becomes intcon (0) semicolon endsy period</pre>
--	---

Pengujian 2

1. Nama Pengujian : Keyword sebagai Prefix Identifier
2. Input file : keyword_identifier_collision.txt
3. Output file : keyword_identifier_collision-result.txt
4. Deskripsi :

Test case ini buat untuk memastikan bahwa apabila keyword dan identifier memiliki awalan yang sama, sehingga hanya lexeme yang persis cocok dengan keyword yang dikenali sebagai keyword, sedangkan sisanya diklasifikasikan sebagai identifier.

5. Hasil Pengujian :

Tabel 4.2 Pengujian 2 : Keyword sebagai Prefix Identifier

Input	Output
<pre> program keywordtest; var programx, beginx, endloop, ifelse, ford, downtox, thensoon, constanta: integer; begin programx := 1; beginx := 2; endloop := 3; ifelse := 4; ford := 5; downtox := 6; thensoon := 7; constanta := 8; end. </pre>	<pre> programsy ident (keywordtest) semicolon varsy ident (programx) comma ident (beginx) comma ident (endloop) comma ident (ifelse) comma ident (ford) comma ident (downtox) comma ident (thensoon) comma ident (constanta) colon ident (integer) semicolon beginsy ident (programx) becomes intcon (1) semicolon ident (beginx) becomes intcon (2) semicolon ident (endloop) becomes </pre>


```
intcon (3)
semicolon
ident (ifelse)
becomes
intcon (4)
semicolon
ident (ford)
becomes
intcon (5)
semicolon
ident (downtox)
becomes
intcon (6)
semicolon
ident (thensoon)
becomes
intcon (7)
semicolon
ident (constant)
becomes
intcon (8)
semicolon
endsy
```

Pengujian 3

1. Nama Pengujian : Escaped Quote
2. Input file : literals.txt
3. Output file : literals-result.txt
4. Deskripsi :

Test case ini ditujukan untuk menguji bahwa lexer dapat mengenali karakter dan string dengan benar, termasuk penanganan escaped quote dan variasi karakter tanpa memecah menjadi token lain.

5. Hasil Pengujian :

Tabel 4.3 Pengujian 3 : Escaped Quote

Input	Output
<pre> program literaltest; var c: char; begin c := 'a'; writeln('hello world'); writeln('it''s valid'); writeln('a,b;c:=d'); writeln('x+y*z/2'); writeln('[](<>'); end. </pre>	<pre> programsy ident (literaltest) semicolon varsy ident (c) colon ident (char) semicolon beginsy ident (c) becomes charcon ('a') semicolon ident (writeln) lparent string ('hello world') rparent semicolon ident (writeln) lparent string ('it''s valid') rparent semicolon ident (writeln) lparent string ('a,b;c:=d') rparent semicolon ident (writeln) lparent string ('x+y*z/2') rparent semicolon ident (writeln) lparent string ('[](<>') rparent semicolon endsy period </pre>

Pengujian 4

1. Nama Pengujian : Comments Format
2. Input file : nested_comments.txt
3. Output file : nested_comments-result.txt
4. Deskripsi :

Test Case ini bertujuan untuk memastikan bahwa lexer dapat menangani bentuk komentar berlapis seperti ({...} dan (*...*)), multiline comments dan karakter { atau } di dalam komentar (* *) atau (* atau *) di dalam komentar { }. Dimana format komentar yang kami adopsi merupakan komentar dengan pembuka dan penutup yang berpasangan,

5. Hasil Pengujian :

Tabel 4.4 Pengujian 4 : Comments Format

Input	Output
<pre>program commenttest; var x: integer; begin x := 1; { outer comment { inner comment } still outer } x := x + 1; (* outer star comment (* inner star comment *) still outer *) x := x + 2; { mix 1 } (* mix 2 *) x := x + 3; end.</pre>	<pre>programsy ident (commenttest) semicolon varsy ident (x) colon ident (integer) semicolon beginsy ident (x) becomes intcon (1) semicolon comment ({ outer comment { inner comment } still outer }) ident (x) becomes ident (x) plus intcon (1) semicolon comment ((* outer star comment (* inner star comment *) still outer *)) ident (x) becomes ident (x) plus intcon (2) semicolon comment ({ mix 1 }) comment ((* mix 2 *)) ident (x) becomes ident (x) plus intcon (3) semicolon endsy period</pre>

Pengujian 5

1. Nama Pengujian : Operator
2. Input file : operators.txt
3. Output file : operators-result.txt
4. Deskripsi :

Test Case ini dilakukan untuk memastikan bahwa lexer dapat mengenali dan mengklasifikasi seluruh operator aritmatika, relasional dan logika sesuai dengan spesifikasi token.

5. Hasil Pengujian :

Tabel 4.5 Pengujian 5 : Operator

Input	Output
<pre>program operatortest; var a, b, c: integer; begin a := 10; b := 3; c := 2; if a == b then a := a + b; if a <> b then a := a - b; if a < b then a := a * b; if a <= b then a := a div b; if a > b then a := a / b; if a >= b then a := a mod b; if not a == b and a >= c or b <= a then c := a + b * c; a := (a + b) * c; b := a / c; end.</pre>	<pre>programsy ident (operatortest) semicolon varsy ident (a) comma ident (b) comma ident (c) colon ident (integer) semicolon beginsy ident (a) becomes intcon (10) semicolon ident (b) becomes intcon (3) semicolon ident (c) becomes intcon (2) semicolon</pre>

```
ifsy
ident (a)
eql
ident (b)
thensy
ident (a)
becomes
ident (a)
plus
ident (b)
semicolon
ifsy
ident (a)
neq
ident (b)
thensy
ident (a)
becomes
ident (a)
minus
ident (b)
semicolon
ifsy
ident (a)
lss
ident (b)
thensy
ident (a)
becomes
ident (a)
times
ident (b)
```

```
semicolon  
ifsy  
ident (a)  
leq  
ident (b)  
thensy  
ident (a)  
becomes  
ident (a)  
idiv  
ident (b)  
semicolon  
ifsy  
ident (a)  
gtr  
ident (b)  
thensy  
ident (a)  
becomes  
ident (a)  
rdiv  
ident (b)  
semicolon  
ifsy  
ident (a)  
geq  
ident (b)  
thensy  
ident (a)  
becomes  
ident (a)  
imod  
ident (b)  
semicolon
```

```
ifsy
notsy
ident (a)
eql
ident (b)
andsy
ident (a)
geq
ident (c)
orsy
ident (b)
leq
ident (a)
thensy
ident (c)
becomes
ident (a)
plus
ident (b)
times
ident (c)
semicolon
```

```
ident (a)
becomes
lparent
ident (a)
plus
ident (b)
rparent
times
ident (c)
semicolon
ident (b)
becomes
ident (a)
rdiv
ident (c)
semicolon
endsy
period
```


Bab 5

Kesimpulan dan Saran

5.1. Kesimpulan

Berdasarkan hasil perancangan dan implementasi yang telah dilakukan, dapat disimpulkan bahwa:

- Program *Lexical Analyzer* berhasil diimplementasikan menggunakan bahasa pemrograman C++ tanpa bantuan *library* generator otomatis.
- Program mampu memproses masukan karakter demi karakter dan melakukan transisi antar *state* sesuai dengan logika DFA yang telah dirancang.
- Seluruh 52 jenis token dapat dikenali dan diklasifikasikan dengan benar oleh program.
- Program berhasil menghasilkan *error* yang sesuai jika tidak mengenali token tersebut.

5.2. Saran

Beberapa hal yang dapat dikembangkan lebih lanjut untuk meningkatkan kualitas program pada pengembangan selanjutnya adalah:

- Perlunya sinkronisasi informasi pada spesifikasi tugas besar sebagai referensi tunggal untuk menjaga validitas instruksi dan efisiensi pencarian jawaban.
- Melakukan evaluasi ulang terhadap diagram transisi DFA yang telah dirancang untuk memastikan tidak ada *state* yang redundan sehingga program dapat memproses lebih optimal.

Lampiran

Tautan Repositori Github:

<https://github.com/Irvin-Tandiarrang-Sumual/STI-Tubes-IF2224-2026.git>

Tautan *Workspace Diagram*:

<https://drive.google.com/file/d/16iKH4CMwHPyhwm0ALfyn3FKYAxC9EKYj/view?usp=sharing>

NIM	Nama	Deskripsi Tugas	Persentase Kontribusi
13524008	Agatha Tatianingseto	- Merancang set testing dan expected output - Fixing error - Fixing comment token	20%
13524030	Irvin Tandiarrang Sumual	- Implementasi lexer - Implementasi token - Implementasi code location - Lead the team	20%
13524074	Bernhard Aprillio Pramana	- Implementasi reader - Fix lexer process string - Menyusun laporan bab 1	20%
13524096	Moreno Syawali Ganda Sugita	- Menyusun laporan - Implementasi output - Fix UNKNOWN token	20%
13524110	Jennifer Khang	- Merancang DFA - Bantu testing - Fix process number	20%

Daftar Referensi

- [1] Tutorialspoint, "Compiler Design - Lexical Analysis." [Online]. Available: https://www.tutorialspoint.com/compiler_design/compiler_design_lexical_analysis.htm. Accessed 24 March 2026.
- [2] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Boston, MA, USA: Addison-Wesley, 2006. [Online]. Available: [https://repository.unikom.ac.id/48769/1/Compilers%20-%20Principles,%20Techniques,%20and%20Tools%20\(2006\).pdf](https://repository.unikom.ac.id/48769/1/Compilers%20-%20Principles,%20Techniques,%20and%20Tools%20(2006).pdf). Accessed 24 March 2026.
- [3] J. E. Hopcroft, R. Motwani, and J. D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 3rd ed. Boston, MA, USA: Addison-Wesley, 2007. [Online]. Available: https://cdn-edunex.itb.ac.id/29161-Formal-Language-Theory-and-Automata/1629640939613_Introduction-to-Automata-Theory.-Languages.-and-Computation-Edition-3.pdf. Accessed 24 March 2026.